

40 Must-Do LeetCode Problems

Statements · Explanations · Java Solutions

@make_it_worth19 · Interview Preparation Guide

Table of Contents

1.	#4	Median of Two Sorted Arrays	S+ — Extremely Hard
2.	#42	Trapping Rain Water	S+ — Extremely Hard
3.	#51	N-Queens	S+ — Extremely Hard
4.	#76	Minimum Window Substring	S+ — Extremely Hard
5.	#127	Word Ladder	S+ — Extremely Hard
6.	#312	Burst Balloons	S — Very Hard
7.	#10	Regular Expression Matching	S — Very Hard
8.	#87	Scramble String	S — Very Hard
9.	#1091	Shortest Path in Binary Matrix	S — Very Hard
10.	#239	Sliding Window Maximum	S — Very Hard
11.	#5	Longest Palindromic Substring	A+ — Hard
12.	#44	Wildcard Matching	A+ — Hard
13.	#241	Different Ways to Add Parentheses	A+ — Hard
14.	#72	Edit Distance	A+ — Hard
15.	#282	Expression Add Operators	A+ — Hard
16.	#15	3Sum	A — Medium (Harder)
17.	#18	4Sum	A — Medium (Harder)
18.	#124	Binary Tree Maximum Path Sum	A — Medium (Harder)
19.	#297	Serialize and Deserialize Binary Tree	A — Medium (Harder)
20.	#210	Course Schedule II	A — Medium (Harder)
21.	#56	Merge Intervals	B — Medium (Mid)
22.	#347	Top K Frequent Elements	B — Medium (Mid)
23.	#215	Kth Largest Element in an Array	B — Medium (Mid)
24.	#98	Validate Binary Search Tree	B — Medium (Mid)
25.	#200	Number of Islands	B — Medium (Mid)
26.	#49	Group Anagrams	C — Medium (Easier)
27.	#121	Best Time to Buy and Sell Stock	C — Medium (Easier)
28.	#3	Longest Substring Without Repeating Characters	C — Medium (Easier)
29.	#560	Subarray Sum Equals K	C — Medium (Easier)
30.	#75	Sort Colors	C — Medium (Easier)
31.	#704	Binary Search	D — Easy (Harder)
32.	#226	Invert Binary Tree	D — Easy (Harder)
33.	#100	Same Tree	D — Easy (Harder)
34.	#104	Maximum Depth of Binary Tree	D — Easy (Harder)
35.	#20	Valid Parentheses	D — Easy (Harder)
36.	#1	Two Sum	E — Easy
37.	#9	Palindrome Number	E — Easy
38.	#206	Reverse Linked List	E — Easy
39.	#217	Contains Duplicate	E — Easy
40.	#141	Linked List Cycle	E — Easy

1. Median of Two Sorted Arrays

LeetCode #4

Binary Search

PROBLEM STATEMENT

Given two sorted arrays `nums1` and `nums2` of size `m` and `n` respectively, return the median of the two sorted arrays. The overall run time complexity should be $O(\log(m+n))$.

APPROACH & EXPLANATION

Binary search on the smaller array to find a partition point such that every element on the left side of the combined partition is less than or equal to every element on the right side. Let the smaller array be `A` (size `m`) and the larger be `B` (size `n`). We binary search a cut `i` in `A` ($0..m$); the corresponding cut in `B` is `j = (m+n+1)/2 - i`. We need `Aleft ≤ Bright` and `Bleft ≤ Aright`. Adjust the binary search bounds until this condition holds, then compute the median from the boundary elements. This avoids merging and runs in $O(\log(\min(m,n)))$.

JAVA SOLUTION

```
class Solution {
    public double findMedianSortedArrays(int[] nums1, int[] nums2) {
        if (nums1.length > nums2.length) return findMedianSortedArrays(nums2, nums1);
        int m = nums1.length, n = nums2.length;
        int lo = 0, hi = m, half = (m + n + 1) / 2;
        while (lo <= hi) {
            int i = (lo + hi) / 2;    // partition in nums1
            int j = half - i;        // partition in nums2

            int aLeft  = (i == 0) ? Integer.MIN_VALUE : nums1[i - 1];
            int aRight = (i == m) ? Integer.MAX_VALUE : nums1[i];
            int bLeft  = (j == 0) ? Integer.MIN_VALUE : nums2[j - 1];
            int bRight = (j == n) ? Integer.MAX_VALUE : nums2[j];

            if (aLeft <= bRight && bLeft <= aRight) {
                if ((m + n) % 2 == 1) return Math.max(aLeft, bLeft);
                return (Math.max(aLeft, bLeft) + Math.min(aRight, bRight)) / 2.0;
            } else if (aLeft > bRight) {
                hi = i - 1;
            } else {
                lo = i + 1;
            }
        }
        throw new IllegalArgumentException("Input arrays are not sorted");
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def findMedianSortedArrays(self, nums1: list[int], nums2: list[int]) -> float:
        if len(nums1) > len(nums2):
            nums1, nums2 = nums2, nums1
        m, n = len(nums1), len(nums2)
        lo, hi, half = 0, m, (m + n + 1) // 2

        while lo <= hi:
            i = (lo + hi) // 2
            j = half - i

            a_left = float('-inf') if i == 0 else nums1[i - 1]
            a_right = float('inf') if i == m else nums1[i]
            b_left = float('-inf') if j == 0 else nums2[j - 1]
            b_right = float('inf') if j == n else nums2[j]

            if a_left <= b_right and b_left <= a_right:
                if (m + n) % 2 == 1:
                    return max(a_left, b_left)
                return (max(a_left, b_left) + min(a_right, b_right)) / 2.0
            elif a_left > b_right:
                hi = i - 1
            else:
                lo = i + 1
        raise ValueError("Input arrays are not sorted")
```

2. Trapping Rain Water

LeetCode #42

Two Pointers / Stack

PROBLEM STATEMENT

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

APPROACH & EXPLANATION

The water trapped above index i equals $\min(\text{maxLeft}[i], \text{maxRight}[i]) - \text{height}[i]$. Instead of precomputing two arrays, use two pointers from both ends with two running maxima (leftMax , rightMax). Whichever side has the smaller running max determines how much water can be trapped at that pointer, because the smaller max is the limiting wall. Move the pointer with the smaller max inward, adding the trapped water as you go. This achieves $O(n)$ time and $O(1)$ extra space.

JAVA SOLUTION

```
class Solution {
    public int trap(int[] height) {
        int left = 0, right = height.length - 1;
        int leftMax = 0, rightMax = 0, water = 0;
        while (left < right) {
            if (height[left] <= height[right]) {
                leftMax = Math.max(leftMax, height[left]);
                water += leftMax - height[left];
                left++;
            } else {
                rightMax = Math.max(rightMax, height[right]);
                water += rightMax - height[right];
                right--;
            }
        }
        return water;
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def trap(self, height: list[int]) -> int:
        left, right = 0, len(height) - 1
        left_max = right_max = water = 0

        while left < right:
            if height[left] <= height[right]:
                left_max = max(left_max, height[left])
                water += left_max - height[left]
                left += 1
            else:
                right_max = max(right_max, height[right])
                water += right_max - height[right]
                right -= 1
        return water
```

3. N-Queens

LeetCode #51

Backtracking

PROBLEM STATEMENT

The n-queens puzzle is the problem of placing n queens on an n x n chessboard such that no two queens attack each other. Return all distinct solutions, each as a board configuration.

APPROACH & EXPLANATION

Place queens row by row. For each row, try every column and check whether that column, and both diagonals, are already attacked by a previously placed queen (tracked with boolean sets/arrays for columns, and the two diagonal directions using row-col and row+col as keys). If a placement is safe, place the queen, recurse to the next row, then undo (backtrack) the placement before trying the next column. When all n rows are filled, record the board as a solution.

JAVA SOLUTION

```
class Solution {
    public List<List<String>> solveNQueens(int n) {
        List<List<String>> results = new ArrayList<>();
        char[][] board = new char[n][n];
        for (char[] row : board) Arrays.fill(row, '.');

        boolean[] cols = new boolean[n];
        boolean[] diag1 = new boolean[2 * n]; // row - col + n
        boolean[] diag2 = new boolean[2 * n]; // row + col

        backtrack(0, n, board, cols, diag1, diag2, results);
        return results;
    }

    private void backtrack(int row, int n, char[][] board, boolean[] cols,
                           boolean[] diag1, boolean[] diag2, List<List<String>> results) {
        if (row == n) {
            List<String> solution = new ArrayList<>();
            for (char[] r : board) solution.add(new String(r));
            results.add(solution);
            return;
        }
        for (int col = 0; col < n; col++) {
            int d1 = row - col + n, d2 = row + col;
            if (cols[col] || diag1[d1] || diag2[d2]) continue;

            board[row][col] = 'Q';
            cols[col] = diag1[d1] = diag2[d2] = true;

            backtrack(row + 1, n, board, cols, diag1, diag2, results);

            board[row][col] = '.';
            cols[col] = diag1[d1] = diag2[d2] = false;
        }
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def solveNQueens(self, n: int) -> list[list[str]]:
        results = []
        board = [['.' * n for _ in range(n)]]
        cols, diag1, diag2 = set(), set(), set()

        def backtrack(row):
            if row == n:
                results.append([''.join(r) for r in board])
                return
            for col in range(n):
                if col in cols or (row - col) in diag1 or (row + col) in diag2:
                    continue
                board[row][col] = 'Q'
                cols.add(col); diag1.add(row - col); diag2.add(row + col)

                backtrack(row + 1)

                board[row][col] = '.'
                cols.remove(col); diag1.remove(row - col); diag2.remove(row + col)

        backtrack(0)
        return results
```

4. Minimum Window Substring

 LeetCode #76 Sliding Window

PROBLEM STATEMENT

Given two strings *s* and *t*, return the minimum window substring of *s* such that every character in *t* (including duplicates) is included in the window. Return an empty string if no such substring exists.

APPROACH & EXPLANATION

Use a sliding window with two pointers (left, right). Maintain a frequency map of characters needed from *t* and a counter of how many distinct required characters are currently satisfied in the window. Expand right, and whenever the window satisfies all requirements, try to shrink from the left to find the smallest valid window, updating the best answer as you shrink. This gives $O(|s| + |t|)$ time.

JAVA SOLUTION

```
class Solution {
    public String minWindow(String s, String t) {
        if (s.isEmpty() || t.isEmpty()) return "";

        Map<Character, Integer> need = new HashMap<>();
        for (char c : t.toCharArray()) need.merge(c, 1, Integer::sum);

        int required = need.size();
        int formed = 0;
        Map<Character, Integer> windowCounts = new HashMap<>();

        int left = 0, bestLen = Integer.MAX_VALUE, bestLeft = 0;

        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            windowCounts.merge(c, 1, Integer::sum);

            if (need.containsKey(c) && windowCounts.get(c).intValue() == need.get(c).intValue()) {
                formed++;
            }

            while (formed == required) {
                if (right - left + 1 < bestLen) {
                    bestLen = right - left + 1;
                    bestLeft = left;
                }
                char lc = s.charAt(left);
                windowCounts.put(lc, windowCounts.get(lc) - 1);
                if (need.containsKey(lc) && windowCounts.get(lc) < need.get(lc)) {
                    formed--;
                }
                left++;
            }
        }
        return bestLen == Integer.MAX_VALUE ? "" : s.substring(bestLeft, bestLeft + bestLen);
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def minWindow(self, s: str, t: str) -> str:
        if not s or not t:
            return ""

        need = Counter(t)
        required = len(need)
        window_counts = {}
        formed = 0
        left = 0
        best_len, best_left = float('inf'), 0

        for right, c in enumerate(s):
            window_counts[c] = window_counts.get(c, 0) + 1
            if c in need and window_counts[c] == need[c]:
                formed += 1

            while formed == required:
                if right - left + 1 < best_len:
                    best_len, best_left = right - left + 1, left
                lc = s[left]
                window_counts[lc] -= 1
                if lc in need and window_counts[lc] < need[lc]:
                    formed -= 1
                left += 1

        return "" if best_len == float('inf') else s[best_left:best_left + best_len]
```

5. Word Ladder

LeetCode #127

BFS

PROBLEM STATEMENT

Given a beginWord, an endWord, and a wordList, return the number of words in the shortest transformation sequence from beginWord to endWord, changing only one letter at a time, with every intermediate word existing in wordList. Return 0 if no such sequence exists.

APPROACH & EXPLANATION

Model each word as a node; an edge exists between two words that differ by exactly one letter. The shortest transformation sequence is the shortest path in this unweighted graph, so BFS is the right tool. Instead of building the full graph explicitly, generate neighbors on the fly: for each word, try changing each position to every letter a-z and check membership in the word set. BFS level by level, tracking the number of transformations, and remove visited words from the set to avoid revisiting.

JAVA SOLUTION

```
class Solution {
    public int ladderLength(String beginWord, String endWord, List<String> wordList) {
        Set<String> dict = new HashSet<>(wordList);
        if (!dict.contains(endWord)) return 0;

        Queue<String> queue = new LinkedList<>();
        queue.offer(beginWord);
        dict.remove(beginWord);
        int steps = 1;

        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                String word = queue.poll();
                if (word.equals(endWord)) return steps;

                char[] chars = word.toCharArray();
                for (int pos = 0; pos < chars.length; pos++) {
                    char original = chars[pos];
                    for (char c = 'a'; c <= 'z'; c++) {
                        if (c == original) continue;
                        chars[pos] = c;
                        String next = new String(chars);
                        if (dict.contains(next)) {
                            dict.remove(next);
                            queue.offer(next);
                        }
                    }
                    chars[pos] = original;
                }
            }
            steps++;
        }
        return 0;
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def ladderLength(self, beginWord: str, endWord: str, wordList: list[str]) -> int:
        word_set = set(wordList)
        if endWord not in word_set:
            return 0

        queue = deque([beginWord])
        word_set.discard(beginWord)
        steps = 1

        while queue:
            for _ in range(len(queue)):
                word = queue.popleft()
                if word == endWord:
                    return steps
                for i in range(len(word)):
                    for c in 'abcdefghijklmnopqrstuvwxyz':
                        if c == word[i]:
                            continue
                        next_word = word[:i] + c + word[i + 1:]
                        if next_word in word_set:
                            word_set.remove(next_word)
                            queue.append(next_word)

            steps += 1
        return 0
```

6. Burst Balloons

LeetCode #312

Interval DP

PROBLEM STATEMENT

You are given n balloons, indexed 0 to $n-1$, each with a number on it. You are asked to burst all the balloons. If you burst balloon i , you get $\text{nums}[\text{left}] * \text{nums}[i] * \text{nums}[\text{right}]$ coins, where left and right are the adjacent (still existing) indices. Return the maximum coins you can collect.

APPROACH & EXPLANATION

Think in reverse: instead of asking "which balloon do I burst first", ask "which balloon do I burst LAST within a sub-range". Pad the array with 1 on both ends. Define $\text{dp}[\text{l}][\text{r}]$ as the max coins from bursting all balloons strictly between indices l and r (exclusive boundaries which remain as walls). For every k between l and r , if balloon k is the last one burst in that range, its neighbors at burst time are exactly $\text{nums}[\text{l}]$ and $\text{nums}[\text{r}]$ (since everything else in between is already gone), so $\text{dp}[\text{l}][\text{r}] = \max \text{ over } k \text{ of } \text{dp}[\text{l}][k] + \text{dp}[k][\text{r}] + \text{nums}[\text{l}] * \text{nums}[k] * \text{nums}[\text{r}]$. Fill by increasing range length. This is classic interval DP, $O(n^3)$ time.

JAVA SOLUTION

```
class Solution {
    public int maxCoins(int[] nums) {
        int n = nums.length;
        int[] balloons = new int[n + 2];
        balloons[0] = balloons[n + 1] = 1;
        for (int i = 0; i < n; i++) balloons[i + 1] = nums[i];

        int[][] dp = new int[n + 2][n + 2];

        for (int len = 2; len < n + 2; len++) {
            for (int l = 0; l + len < n + 2; l++) {
                int r = l + len;
                for (int k = l + 1; k < r; k++) {
                    int coins = balloons[l] * balloons[k] * balloons[r]
                        + dp[l][k] + dp[k][r];
                    dp[l][r] = Math.max(dp[l][r], coins);
                }
            }
        }
        return dp[0][n + 1];
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def maxCoins(self, nums: list[int]) -> int:
        balloons = [1] + nums + [1]
        n = len(balloons)
        dp = [[0] * (n + 2) for _ in range(n + 2)]

        for length in range(2, n + 2):
            for l in range(0, n + 2 - length):
                r = l + length
                for k in range(l + 1, r):
                    coins = balloons[l] * balloons[k] * balloons[r] + dp[l][k] + dp[k][r]
                    dp[l][r] = max(dp[l][r], coins)
        return dp[0][n + 1]
```


7. Regular Expression Matching

LeetCode #10

DP

PROBLEM STATEMENT

Given an input string *s* and a pattern *p*, implement regular expression matching that supports '.' (matches any single character) and '*' (matches zero or more of the preceding element). The match should cover the entire input string.

APPROACH & EXPLANATION

Let $dp[i][j]$ mean whether $s[0..i]$ matches $p[0..j]$. Base case $dp[0][0] = \text{true}$. If $p[j-1]$ is not '*', $dp[i][j] = dp[i-1][j-1]$ AND ($s[i-1]$ matches $p[j-1]$, where '.' matches anything). If $p[j-1]$ is '*', it pairs with $p[j-2]$: $dp[i][j]$ is true either if we ignore the "x*" pair entirely ($dp[i][j-2]$), or, if the preceding character matches $s[i-1]$, we can also use $dp[i-1][j]$ (consuming one more character of *s* while keeping the same "x*" available). Fill the table bottom-up in $O(m*n)$ time and space.

JAVA SOLUTION

```
class Solution {
    public boolean isMatch(String s, String p) {
        int m = s.length(), n = p.length();
        boolean[][] dp = new boolean[m + 1][n + 1];
        dp[0][0] = true;

        for (int j = 1; j <= n; j++) {
            if (p.charAt(j - 1) == '*') dp[0][j] = dp[0][j - 2];
        }

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                char pc = p.charAt(j - 1);
                if (pc == '*') {
                    dp[i][j] = dp[i][j - 2]; // zero occurrences
                    char prev = p.charAt(j - 2);
                    if (prev == '.' || prev == s.charAt(i - 1)) {
                        dp[i][j] = dp[i][j] || dp[i - 1][j];
                    }
                } else if (pc == '.' || pc == s.charAt(i - 1)) {
                    dp[i][j] = dp[i - 1][j - 1];
                }
            }
        }
        return dp[m][n];
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        m, n = len(s), len(p)
        dp = [[False] * (n + 1) for _ in range(m + 1)]
        dp[0][0] = True

        for j in range(1, n + 1):
            if p[j - 1] == '*':
                dp[0][j] = dp[0][j - 2]

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                pc = p[j - 1]
                if pc == '*':
                    dp[i][j] = dp[i][j - 2]
                    prev = p[j - 2]
                    if prev == '.' or prev == s[i - 1]:
                        dp[i][j] = dp[i][j] or dp[i - 1][j]
                elif pc == '.' or pc == s[i - 1]:
                    dp[i][j] = dp[i - 1][j - 1]

        return dp[m][n]
```

8. Scramble String

LeetCode #87

DP (String)

PROBLEM STATEMENT

Given two strings `s1` and `s2` of the same length, determine if `s2` is a scrambled string of `s1`. A scrambled string is produced by recursively splitting the string into two non-empty substrings and optionally swapping them, repeated any number of times.

APPROACH & EXPLANATION

Use memoized recursion on (`s1`, `s2`) (or interval indices). Quick prunes: if `s1 == s2`, true; if the sorted characters differ, false (can't be a scramble). Otherwise, try every split point `k`: either the left parts match and the right parts match recursively (no swap: `isScramble(s1[0:k], s2[0:k])` and `isScramble(s1[k:], s2[k:])`), or the split is swapped (`isScramble(s1[0:k], s2[len-k:]` and `isScramble(s1[k:], s2[0:len-k])`). If any split works, the strings are scrambles of each other. Memoize on the substring pair to avoid recomputation — $O(n^4)$ with memoization.

JAVA SOLUTION

```
class Solution {
    private Map<String, Boolean> memo = new HashMap<>();

    public boolean isScramble(String s1, String s2) {
        if (s1.equals(s2)) return true;
        if (s1.length() != s2.length()) return false;

        String key = s1 + "#" + s2;
        if (memo.containsKey(key)) return memo.get(key);

        int[] freq = new int[26];
        for (int i = 0; i < s1.length(); i++) {
            freq[s1.charAt(i) - 'a']++;
            freq[s2.charAt(i) - 'a']--;
        }
        for (int f : freq) {
            if (f != 0) {
                memo.put(key, false);
                return false;
            }
        }

        int n = s1.length();
        for (int k = 1; k < n; k++) {
            boolean noSwap = isScramble(s1.substring(0, k), s2.substring(0, k))
                && isScramble(s1.substring(k), s2.substring(k));
            if (noSwap) { memo.put(key, true); return true; }

            boolean swap = isScramble(s1.substring(0, k), s2.substring(n - k))
                && isScramble(s1.substring(k), s2.substring(0, n - k));
            if (swap) { memo.put(key, true); return true; }
        }
        memo.put(key, false);
        return false;
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def isScramble(self, s1: str, s2: str) -> bool:
        memo = {}

        def helper(a, b):
            if a == b:
                return True
            if len(a) != len(b) or Counter(a) != Counter(b):
                return False
            key = (a, b)
            if key in memo:
                return memo[key]

            n = len(a)
            for k in range(1, n):
                if helper(a[:k], b[:k]) and helper(a[k:], b[k:]):
                    memo[key] = True
                    return True
                if helper(a[:k], b[n - k:]) and helper(a[k:], b[:n - k]):
                    memo[key] = True
                    return True
            memo[key] = False
            return False

        return helper(s1, s2)
```

9. Shortest Path in Binary Matrix

LeetCode #1091

BFS

PROBLEM STATEMENT

Given an $n \times n$ binary matrix grid, return the length of the shortest clear path from top-left to bottom-right (8-directionally connected, moving only through cells with value 0). Return -1 if there is no such path.

APPROACH & EXPLANATION

This is a shortest-path-in-unweighted-graph problem, so BFS is optimal. Each cell is a node, and edges connect it to its up-to-8 neighbors that are within bounds and have value 0. Start BFS from (0,0) if it is 0, track distance level by level, and mark cells visited as soon as they are enqueued (not when dequeued) to avoid enqueueing the same cell multiple times. Return the distance when the bottom-right cell is reached, else -1.

JAVA SOLUTION

```
class Solution {
    public int shortestPathBinaryMatrix(int[][] grid) {
        int n = grid.length;
        if (grid[0][0] != 0 || grid[n - 1][n - 1] != 0) return -1;

        int[][] dirs = {{-1,-1},{-1,0},{-1,1},{0,-1},{0,1},{1,-1},{1,0},{1,1}};
        Queue<int[]> queue = new LinkedList<>();
        queue.offer(new int[]{0, 0});
        grid[0][0] = 1; // reuse grid as visited marker

        int dist = 1;
        while (!queue.isEmpty()) {
            int size = queue.size();
            for (int i = 0; i < size; i++) {
                int[] cur = queue.poll();
                int r = cur[0], c = cur[1];
                if (r == n - 1 && c == n - 1) return dist;

                for (int[] d : dirs) {
                    int nr = r + d[0], nc = c + d[1];
                    if (nr >= 0 && nr < n && nc >= 0 && nc < n && grid[nr][nc] == 0) {
                        grid[nr][nc] = 1;
                        queue.offer(new int[]{nr, nc});
                    }
                }
            }
            dist++;
        }
        return -1;
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def shortestPathBinaryMatrix(self, grid: list[list[int]]) -> int:
        n = len(grid)
        if grid[0][0] != 0 or grid[n - 1][n - 1] != 0:
            return -1

        dirs = [(-1,-1), (-1,0), (-1,1), (0,-1), (0,1), (1,-1), (1,0), (1,1)]
        queue = deque([(0, 0)])
        grid[0][0] = 1
        dist = 1

        while queue:
            for _ in range(len(queue)):
                r, c = queue.popleft()
                if r == n - 1 and c == n - 1:
                    return dist
                for dr, dc in dirs:
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < n and 0 <= nc < n and grid[nr][nc] == 0:
                        grid[nr][nc] = 1
                        queue.append((nr, nc))

            dist += 1
        return -1
```

10. Sliding Window Maximum

LeetCode #239

Monotonic Deque

PROBLEM STATEMENT

You are given an array `nums` and a sliding window of size `k` moving from the very left to the very right. Return an array of the maximum value in each window position.

APPROACH & EXPLANATION

Maintain a deque of indices whose corresponding values are in strictly decreasing order. For each new index `i`: pop indices from the back whose values are smaller than `nums[i]` (they can never be the max again while `nums[i]` is around), then push `i`. Pop from the front if the front index has fallen outside the current window (`i - k`). The front of the deque is always the index of the maximum for the current window. Each index is pushed and popped at most once, giving $O(n)$ total time.

JAVA SOLUTION

```
class Solution {
    public int[] maxSlidingWindow(int[] nums, int k) {
        int n = nums.length;
        int[] result = new int[n - k + 1];
        Deque<Integer> deque = new ArrayDeque<>(); // stores indices, decreasing values

        for (int i = 0; i < n; i++) {
            while (!deque.isEmpty() && deque.peekFirst() <= i - k) {
                deque.pollFirst();
            }
            while (!deque.isEmpty() && nums[deque.peekLast()] < nums[i]) {
                deque.pollLast();
            }
            deque.offerLast(i);

            if (i >= k - 1) {
                result[i - k + 1] = nums[deque.peekFirst()];
            }
        }
        return result;
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def maxSlidingWindow(self, nums: list[int], k: int) -> list[int]:
        dq = deque() # stores indices, decreasing values
        result = []

        for i, num in enumerate(nums):
            while dq and dq[0] <= i - k:
                dq.popleft()
            while dq and nums[dq[-1]] < num:
                dq.pop()
            dq.append(i)

            if i >= k - 1:
                result.append(nums[dq[0]])
        return result
```

11. Longest Palindromic Substring

LeetCode #5 [Expand Around Center](#)

PROBLEM STATEMENT

Given a string *s*, return the longest palindromic substring in *s*.

APPROACH & EXPLANATION

A palindrome mirrors around its center, and a center can either be a single character (odd length) or between two characters (even length) — there are $2n-1$ possible centers. For each center, expand outward as long as the characters on both sides match, tracking the longest expansion found. This avoids the $O(n^2)$ DP table's memory cost while keeping $O(n^2)$ time and $O(1)$ extra space, which is simple and efficient enough in practice.

JAVA SOLUTION

```
class Solution {
    public String longestPalindrome(String s) {
        if (s == null || s.length() < 1) return "";
        int start = 0, end = 0;

        for (int i = 0; i < s.length(); i++) {
            int len1 = expandFromCenter(s, i, i);    // odd length
            int len2 = expandFromCenter(s, i, i + 1); // even length
            int len = Math.max(len1, len2);
            if (len > end - start + 1) {
                start = i - (len - 1) / 2;
                end = i + len / 2;
            }
        }
        return s.substring(start, end + 1);
    }

    private int expandFromCenter(String s, int left, int right) {
        while (left >= 0 && right < s.length() && s.charAt(left) == s.charAt(right)) {
            left--;
            right++;
        }
        return right - left - 1;
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def longestPalindrome(self, s: str) -> str:
        if not s:
            return ""
        start, end = 0, 0

        def expand(left, right):
            while left >= 0 and right < len(s) and s[left] == s[right]:
                left -= 1
                right += 1
            return right - left - 1

        for i in range(len(s)):
            len1 = expand(i, i)
            len2 = expand(i, i + 1)
            length = max(len1, len2)
            if length > end - start + 1:
                start = i - (length - 1) // 2
                end = i + length // 2
        return s[start:end + 1]
```

12. Wildcard Matching

LeetCode #44

DP

PROBLEM STATEMENT

Given an input string s and a pattern p , implement wildcard pattern matching supporting '?' (matches any single character) and '*' (matches any sequence of characters, including the empty sequence). The match should cover the entire input string.

APPROACH & EXPLANATION

$dp[i][j]$ represents whether $s[0..i)$ matches $p[0..j)$. Base cases: $dp[0][0] = \text{true}$; $dp[0][j]$ is true only if $p[0..j)$ is all '*' characters. Transition: if $p[j-1]$ is '?' or equals $s[i-1]$, $dp[i][j] = dp[i-1][j-1]$. If $p[j-1]$ is '*', it can match zero characters ($dp[i][j-1]$) or absorb one more character of s while still being available ($dp[i-1][j]$) — $dp[i][j] = dp[i][j-1] \vee dp[i-1][j]$. Fill bottom-up in $O(m \cdot n)$ time; this can be optimized to $O(n)$ space with a rolling array, but the 2D table is clearest.

JAVA SOLUTION

```
class Solution {
    public boolean isMatch(String s, String p) {
        int m = s.length(), n = p.length();
        boolean[][] dp = new boolean[m + 1][n + 1];
        dp[0][0] = true;

        for (int j = 1; j <= n; j++) {
            if (p.charAt(j - 1) == '*') dp[0][j] = dp[0][j - 1];
        }

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                char pc = p.charAt(j - 1);
                if (pc == '*') {
                    dp[i][j] = dp[i][j - 1] || dp[i - 1][j];
                } else if (pc == '?' || pc == s.charAt(i - 1)) {
                    dp[i][j] = dp[i - 1][j - 1];
                }
            }
        }

        return dp[m][n];
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def isMatch(self, s: str, p: str) -> bool:
        m, n = len(s), len(p)
        dp = [[False] * (n + 1) for _ in range(m + 1)]
        dp[0][0] = True

        for j in range(1, n + 1):
            if p[j - 1] == '*':
                dp[0][j] = dp[0][j - 1]

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                pc = p[j - 1]
                if pc == '*':
                    dp[i][j] = dp[i][j - 1] or dp[i - 1][j]
                elif pc == '?' or pc == s[i - 1]:
                    dp[i][j] = dp[i - 1][j - 1]

        return dp[m][n]
```

13. Different Ways to Add Parentheses

LeetCode #241

Divide and Conquer

PROBLEM STATEMENT

Given a string expression of numbers and operators (+, -, *), return all possible results from computing all the different possible ways to group numbers and operators, by inserting parentheses in every possible position.

APPROACH & EXPLANATION

Divide and conquer: for every operator found in the expression, split the string into a left and a right substring around that operator, recursively compute all possible results for each side, then combine every pair of left and right results using that operator. The base case is a substring with no operator, which is just a single number. Memoization on substrings avoids recomputation for repeated subexpressions.

JAVA SOLUTION

```
class Solution {
    private Map<String, List<Integer>> memo = new HashMap<>();

    public List<Integer> diffWaysToCompute(String expression) {
        if (memo.containsKey(expression)) return memo.get(expression);

        List<Integer> results = new ArrayList<>();
        for (int i = 0; i < expression.length(); i++) {
            char c = expression.charAt(i);
            if (c == '+' || c == '-' || c == '*') {
                List<Integer> left = diffWaysToCompute(expression.substring(0, i));
                List<Integer> right = diffWaysToCompute(expression.substring(i + 1));

                for (int l : left) {
                    for (int r : right) {
                        switch (c) {
                            case '+': results.add(l + r); break;
                            case '-': results.add(l - r); break;
                            case '*': results.add(l * r); break;
                        }
                    }
                }
            }
        }
        if (results.isEmpty()) {
            results.add(Integer.parseInt(expression));
        }
        memo.put(expression, results);
        return results;
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def diffWaysToCompute(self, expression: str) -> list[int]:
        memo = {}

        def compute(expr):
            if expr in memo:
                return memo[expr]
            results = []
            for i, c in enumerate(expr):
                if c in '+-*':
                    left = compute(expr[:i])
                    right = compute(expr[i + 1:])
                    for l in left:
                        for r in right:
                            if c == '+':
                                results.append(l + r)
                            elif c == '-':
                                results.append(l - r)
                            else:
                                results.append(l * r)
            if not results:
                results.append(int(expr))
            memo[expr] = results
            return results

        return compute(expression)
```

14. Edit Distance

LeetCode #72

DP

PROBLEM STATEMENT

Given two strings word1 and word2, return the minimum number of operations (insert, delete, replace a character) required to convert word1 to word2.

APPROACH & EXPLANATION

$dp[i][j]$ = minimum edits to convert the first i characters of word1 into the first j characters of word2. Base cases: $dp[i][0] = i$ (delete all), $dp[0][j] = j$ (insert all). Transition: if the last characters match, $dp[i][j] = dp[i-1][j-1]$ (no operation needed). Otherwise $dp[i][j] = 1 + \min$ of $dp[i-1][j]$ (delete from word1), $dp[i][j-1]$ (insert into word1), $dp[i-1][j-1]$ (replace). Classic bottom-up DP, $O(m*n)$ time and space (can be reduced to $O(n)$ space with a rolling row).

JAVA SOLUTION

```
class Solution {
    public int minDistance(String word1, String word2) {
        int m = word1.length(), n = word2.length();
        int[][] dp = new int[m + 1][n + 1];

        for (int i = 0; i <= m; i++) dp[i][0] = i;
        for (int j = 0; j <= n; j++) dp[0][j] = j;

        for (int i = 1; i <= m; i++) {
            for (int j = 1; j <= n; j++) {
                if (word1.charAt(i - 1) == word2.charAt(j - 1)) {
                    dp[i][j] = dp[i - 1][j - 1];
                } else {
                    dp[i][j] = 1 + Math.min(dp[i - 1][j - 1],
                                            Math.min(dp[i - 1][j], dp[i][j - 1]));
                }
            }
        }
        return dp[m][n];
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def minDistance(self, word1: str, word2: str) -> int:
        m, n = len(word1), len(word2)
        dp = [[0] * (n + 1) for _ in range(m + 1)]

        for i in range(m + 1):
            dp[i][0] = i
        for j in range(n + 1):
            dp[0][j] = j

        for i in range(1, m + 1):
            for j in range(1, n + 1):
                if word1[i - 1] == word2[j - 1]:
                    dp[i][j] = dp[i - 1][j - 1]
                else:
                    dp[i][j] = 1 + min(dp[i - 1][j - 1], dp[i - 1][j], dp[i][j - 1])
        return dp[m][n]
```


15. Expression Add Operators

LeetCode #282

Backtracking

PROBLEM STATEMENT

Given a string `num` that contains only digits and an integer `target`, return all possibilities to insert the binary operators `+`, `-`, and `*` between the digits of `num` so that the resultant expression evaluates to `target`.

APPROACH & EXPLANATION

Backtrack through the string, at each position trying every possible next operand (being careful to skip operands with leading zeros unless the operand itself is "0"). Track the running evaluated value and the value of the last operand separately, because multiplication needs to "undo" the previous addition/subtraction: when appending a `*` operation, the new total is `total - lastOperand + lastOperand * currentOperand`. Explore all three operators at every split point, and when the string is fully consumed, check if the running total equals `target`.

JAVA SOLUTION

```
class Solution {
    public List<String> addOperators(String num, int target) {
        List<String> results = new ArrayList<>();
        if (num == null || num.length() == 0) return results;
        backtrack(results, new StringBuilder(), num, target, 0, 0, 0);
        return results;
    }

    private void backtrack(List<String> results, StringBuilder expr, String num, int target,
                           int pos, long evaluated, long lastOperand) {
        if (pos == num.length()) {
            if (evaluated == target) results.add(expr.toString());
            return;
        }

        for (int i = pos; i < num.length(); i++) {
            if (i > pos && num.charAt(pos) == '0') break; // skip leading-zero operands

            long operand = Long.parseLong(num.substring(pos, i + 1));
            int len = expr.length();

            if (pos == 0) {
                backtrack(results, expr.append(operand), num, target, i + 1, operand, operand);
                expr.setLength(len);
            } else {
                backtrack(results, expr.append("+").append(operand), num, target, i + 1,
                    evaluated + operand, operand);
                expr.setLength(len);

                backtrack(results, expr.append("-").append(operand), num, target, i + 1,
                    evaluated - operand, -operand);
                expr.setLength(len);

                backtrack(results, expr.append("*").append(operand), num, target, i + 1,
                    evaluated - lastOperand + lastOperand * operand, lastOperand * operand);
                expr.setLength(len);
            }
        }
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def addOperators(self, num: str, target: int) -> list[str]:
        results = []
        n = len(num)

        def backtrack(pos, expr, evaluated, last_operand):
            if pos == n:
                if evaluated == target:
                    results.append(expr)
                return
            for i in range(pos, n):
                if i > pos and num[pos] == '0':
                    break
                operand_str = num[pos:i + 1]
                operand = int(operand_str)

                if pos == 0:
                    backtrack(i + 1, operand_str, operand, operand)
                else:
                    backtrack(i + 1, expr + '+' + operand_str, evaluated + operand, operand)
                    backtrack(i + 1, expr + '-' + operand_str, evaluated - operand, -operand)
                    backtrack(i + 1, expr + '*' + operand_str,
                        evaluated - last_operand + last_operand * operand,
                        last_operand * operand)

        if num:
            backtrack(0, "", 0, 0)
        return results
```

16. 3Sum

LeetCode #15

Two Pointers

PROBLEM STATEMENT

Given an integer array `nums`, return all unique triplets `[nums[i], nums[j], nums[k]]` such that `i != j != k` and `nums[i] + nums[j] + nums[k] == 0`.

APPROACH & EXPLANATION

Sort the array first. Fix one element `nums[i]` and use two pointers (`left`, `right`) on the remaining sorted range to find pairs summing to `-nums[i]`, moving `left` up and `right` down as needed. Sorting lets us skip duplicate values for `i`, `left`, and `right` in $O(1)$, avoiding duplicate triplets. Overall $O(n^2)$ time after the $O(n \log n)$ sort.

JAVA SOLUTION

```

class Solution {
    public List<List<Integer>> threeSum(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> result = new ArrayList<>();

        for (int i = 0; i < nums.length - 2; i++) {
            if (i > 0 && nums[i] == nums[i - 1]) continue; // skip duplicate anchors
            if (nums[i] > 0) break; // sorted, no triplet possible

            int left = i + 1, right = nums.length - 1;
            while (left < right) {
                int sum = nums[i] + nums[left] + nums[right];
                if (sum == 0) {
                    result.add(Arrays.asList(nums[i], nums[left], nums[right]));
                    while (left < right && nums[left] == nums[left + 1]) left++;
                    while (left < right && nums[right] == nums[right - 1]) right--;
                    left++;
                    right--;
                } else if (sum < 0) {
                    left++;
                } else {
                    right--;
                }
            }
        }
        return result;
    }
}

```

PYTHON SOLUTION

```
class Solution:
    def threeSum(self, nums: list[int]) -> list[list[int]]:
        nums.sort()
        result = []

        for i in range(len(nums) - 2):
            if i > 0 and nums[i] == nums[i - 1]:
                continue
            if nums[i] > 0:
                break

            left, right = i + 1, len(nums) - 1
            while left < right:
                total = nums[i] + nums[left] + nums[right]
                if total == 0:
                    result.append([nums[i], nums[left], nums[right]])
                    while left < right and nums[left] == nums[left + 1]:
                        left += 1
                    while left < right and nums[right] == nums[right - 1]:
                        right -= 1
                    left += 1
                    right -= 1
                elif total < 0:
                    left += 1
                else:
                    right -= 1

        return result
```


18. Binary Tree Maximum Path Sum

LeetCode #124

Tree / Postorder DFS

PROBLEM STATEMENT

Given the root of a binary tree, return the maximum path sum of any non-empty path. A path is any sequence of nodes connected by edges, where each node appears at most once, and does not need to pass through the root.

APPROACH & EXPLANATION

For each node, compute the maximum "downward" contribution it can offer to a parent (its own value plus the better of its left/right child's contribution, clamped at 0 since negative contributions should be dropped). While computing this bottom-up (postorder), also consider the best path that uses this node as its highest point, i.e. $\text{node.val} + \text{leftContribution} + \text{rightContribution}$, and update a global maximum with that. Return the downward contribution to the caller so the parent can use it.

JAVA SOLUTION

```
class Solution {
    private int maxSum = Integer.MIN_VALUE;

    public int maxPathSum(TreeNode root) {
        maxDownward(root);
        return maxSum;
    }

    private int maxDownward(TreeNode node) {
        if (node == null) return 0;

        int leftGain = Math.max(maxDownward(node.left), 0);
        int rightGain = Math.max(maxDownward(node.right), 0);

        int priceThroughNode = node.val + leftGain + rightGain;
        maxSum = Math.max(maxSum, priceThroughNode);

        return node.val + Math.max(leftGain, rightGain);
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def maxPathSum(self, root: Optional[TreeNode]) -> int:
        self.max_sum = float('-inf')

        def max_downward(node):
            if not node:
                return 0
            left_gain = max(max_downward(node.left), 0)
            right_gain = max(max_downward(node.right), 0)

            price_through_node = node.val + left_gain + right_gain
            self.max_sum = max(self.max_sum, price_through_node)

            return node.val + max(left_gain, right_gain)

        max_downward(root)
        return self.max_sum
```

19. Serialize and Deserialize Binary Tree

LeetCode #297

Tree

PROBLEM STATEMENT

Design an algorithm to serialize a binary tree to a string and deserialize that string back to the original tree structure.

APPROACH & EXPLANATION

Use preorder traversal for serialization: write the current node's value, then recursively serialize left, then right. Use a sentinel marker (e.g. "#") for null children, and a delimiter (e.g. ",") between values, producing a string that unambiguously encodes structure. For deserialization, split the string into tokens and rebuild the tree by consuming tokens in the same preorder sequence: read a token, if it's the null marker return null, otherwise create a node and recursively build its left and right subtrees from the remaining tokens.

JAVA SOLUTION

```
public class Codec {

    public String serialize(TreeNode root) {
        StringBuilder sb = new StringBuilder();
        buildString(root, sb);
        return sb.toString();
    }

    private void buildString(TreeNode node, StringBuilder sb) {
        if (node == null) {
            sb.append("#,");
        } else {
            sb.append(node.val).append(",");
            buildString(node.left, sb);
            buildString(node.right, sb);
        }
    }

    public TreeNode deserialize(String data) {
        Deque<String> tokens = new LinkedList<>(Arrays.asList(data.split(",")));
        return buildTree(tokens);
    }

    private TreeNode buildTree(Deque<String> tokens) {
        String val = tokens.poll();
        if (val.equals("#")) return null;

        TreeNode node = new TreeNode(Integer.parseInt(val));
        node.left = buildTree(tokens);
        node.right = buildTree(tokens);
        return node;
    }
}
```

PYTHON SOLUTION

```
class Codec:
    def serialize(self, root: Optional[TreeNode]) -> str:
        parts = []

        def build(node):
            if not node:
                parts.append('#')
            else:
                parts.append(str(node.val))
                build(node.left)
                build(node.right)

        build(root)
        return ','.join(parts)

    def deserialize(self, data: str) -> Optional[TreeNode]:
        tokens = iter(data.split(','))

        def build():
            val = next(tokens)
            if val == '#':
                return None
            node = TreeNode(int(val))
            node.left = build()
            node.right = build()
            return node

        return build()
```

20. Course Schedule II

LeetCode #210

Topological Sort

PROBLEM STATEMENT

There are `numCourses` courses labeled 0 to `numCourses-1`. Given prerequisite pairs, return an ordering of courses to finish all courses, or an empty array if it is impossible (a cycle exists).

APPROACH & EXPLANATION

Model courses as nodes and prerequisites as directed edges. Compute in-degree for every node and build an adjacency list. Use Kahn's algorithm: push all nodes with in-degree 0 onto a queue, repeatedly pop a node, add it to the result order, and decrement the in-degree of its neighbors, pushing any that reach 0. If the result contains all `numCourses` nodes, it is a valid topological order; otherwise a cycle exists and no valid ordering is possible, so return an empty array.

JAVA SOLUTION

```
class Solution {
    public int[] findOrder(int numCourses, int[][] prerequisites) {
        List<List<Integer>> graph = new ArrayList<>();
        for (int i = 0; i < numCourses; i++) graph.add(new ArrayList<>());

        int[] inDegree = new int[numCourses];
        for (int[] p : prerequisites) {
            int course = p[0], prereq = p[1];
            graph.get(prereq).add(course);
            inDegree[course]++;
        }

        Queue<Integer> queue = new LinkedList<>();
        for (int i = 0; i < numCourses; i++) {
            if (inDegree[i] == 0) queue.offer(i);
        }

        int[] order = new int[numCourses];
        int idx = 0;

        while (!queue.isEmpty()) {
            int course = queue.poll();
            order[idx++] = course;
            for (int next : graph.get(course)) {
                if (--inDegree[next] == 0) queue.offer(next);
            }
        }
        return idx == numCourses ? order : new int[0];
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def findOrder(self, numCourses: int, prerequisites: list[list[int]]) -> list[int]:
        graph = [[] for _ in range(numCourses)]
        in_degree = [0] * numCourses

        for course, prereq in prerequisites:
            graph[prereq].append(course)
            in_degree[course] += 1

        queue = deque([i for i in range(numCourses) if in_degree[i] == 0])
        order = []

        while queue:
            course = queue.popleft()
            order.append(course)
            for nxt in graph[course]:
                in_degree[nxt] -= 1
                if in_degree[nxt] == 0:
                    queue.append(nxt)

        return order if len(order) == numCourses else []
```

21. Merge Intervals

LeetCode #56

Sort + Merge

PROBLEM STATEMENT

Given an array of intervals where $\text{intervals}[i] = [\text{start}_i, \text{end}_i]$, merge all overlapping intervals and return an array of the non-overlapping intervals that cover all the intervals in the input.

APPROACH & EXPLANATION

Sort intervals by start time. Iterate through them keeping a "current merged interval". If the next interval's start is less than or equal to the current interval's end, they overlap, so extend the current interval's end to the max of both ends. Otherwise, the current interval is finalized and a new current interval begins. This greedy sweep runs in $O(n \log n)$ due to sorting.

JAVA SOLUTION

```
class Solution {
    public int[][] merge(int[][] intervals) {
        Arrays.sort(intervals, (a, b) -> Integer.compare(a[0], b[0]));

        List<int[]> merged = new ArrayList<>();
        for (int[] interval : intervals) {
            if (merged.isEmpty() || merged.get(merged.size() - 1)[1] < interval[0]) {
                merged.add(interval);
            } else {
                merged.get(merged.size() - 1)[1] =
                    Math.max(merged.get(merged.size() - 1)[1], interval[1]);
            }
        }
        return merged.toArray(new int[merged.size()][]);
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def merge(self, intervals: list[list[int]]) -> list[list[int]]:
        intervals.sort(key=lambda x: x[0])
        merged = []

        for interval in intervals:
            if not merged or merged[-1][1] < interval[0]:
                merged.append(interval)
            else:
                merged[-1][1] = max(merged[-1][1], interval[1])
        return merged
```

22. Top K Frequent Elements

LeetCode #347

Min Heap / Frequency Map

PROBLEM STATEMENT

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements.

APPROACH & EXPLANATION

Count the frequency of every number using a hash map. Use a min-heap of size `k` keyed by frequency: push each (number, frequency) pair, and whenever the heap grows past size `k`, pop the smallest-frequency element. What remains in the heap after processing all entries is the top `k` frequent elements. This runs in $O(n \log k)$ time, better than sorting all elements when `k` is small. (Bucket sort by frequency achieves $O(n)$ but the heap approach is more broadly applicable.)

JAVA SOLUTION

```
class Solution {
    public int[] topKFrequent(int[] nums, int k) {
        Map<Integer, Integer> freq = new HashMap<>();
        for (int num : nums) freq.merge(num, 1, Integer::sum);

        PriorityQueue<Map.Entry<Integer, Integer>> minHeap =
            new PriorityQueue<>((a, b) -> a.getValue() - b.getValue());

        for (Map.Entry<Integer, Integer> entry : freq.entrySet()) {
            minHeap.offer(entry);
            if (minHeap.size() > k) minHeap.poll();
        }

        int[] result = new int[k];
        for (int i = k - 1; i >= 0; i--) {
            result[i] = minHeap.poll().getKey();
        }
        return result;
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def topKFrequent(self, nums: list[int], k: int) -> list[int]:
        freq = Counter(nums)
        heap = []

        for num, count in freq.items():
            heapq.heappush(heap, (count, num))
            if len(heap) > k:
                heapq.heappop(heap)

        return [num for count, num in heap]
```


23. Kth Largest Element in an Array

LeetCode #215

Quickselect / Min Heap

PROBLEM STATEMENT

Find the kth largest element in an unsorted array. Note that it is the kth largest element in sorted order, not the kth distinct element.

APPROACH & EXPLANATION

Two common approaches: (1) Maintain a min-heap of size k ; iterate through the array, push each element, and pop when size exceeds k — the heap's top is the kth largest at the end, $O(n \log k)$. (2) Quickselect: like quicksort's partition step, repeatedly partition around a pivot and recurse only into the side that must contain the kth largest, giving average $O(n)$ time. The heap version is shown below for simplicity and guaranteed $O(n \log k)$ worst case.

JAVA SOLUTION

```
class Solution {
    public int findKthLargest(int[] nums, int k) {
        PriorityQueue<Integer> minHeap = new PriorityQueue<>();
        for (int num : nums) {
            minHeap.offer(num);
            if (minHeap.size() > k) minHeap.poll();
        }
        return minHeap.peek();
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def findKthLargest(self, nums: list[int], k: int) -> int:
        heap = []
        for num in nums:
            heapq.heappush(heap, num)
            if len(heap) > k:
                heapq.heappop(heap)
        return heap[0]
```

24. Validate Binary Search Tree

LeetCode #98

Tree

PROBLEM STATEMENT

Given the root of a binary tree, determine if it is a valid binary search tree (BST), where every node's value lies strictly within the open range defined by its ancestors.

APPROACH & EXPLANATION

A common mistake is only comparing a node with its immediate children — that misses cases where a deep node violates an ancestor further up. Instead, recursively pass down a valid (min, max) range for each node: the root's range is $(-\infty, \infty)$; when recursing left, the max tightens to the current node's value; when recursing right, the min tightens to the current node's value. A node is valid only if its value strictly lies within its passed-down range, and both children must also be valid under their updated ranges.

JAVA SOLUTION

```
class Solution {
    public boolean isValidBST(TreeNode root) {
        return validate(root, null, null);
    }

    private boolean validate(TreeNode node, Long lower, Long upper) {
        if (node == null) return true;

        if (lower != null && node.val <= lower) return false;
        if (upper != null && node.val >= upper) return false;

        return validate(node.left, lower, (long) node.val)
            && validate(node.right, (long) node.val, upper);
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def isValidBST(self, root: Optional[TreeNode]) -> bool:
        def validate(node, lower, upper):
            if not node:
                return True
            if lower is not None and node.val <= lower:
                return False
            if upper is not None and node.val >= upper:
                return False
            return validate(node.left, lower, node.val) and validate(node.right, node.val, upper)

        return validate(root, None, None)
```

25. Number of Islands

LeetCode #200

DFS / Flood Fill

PROBLEM STATEMENT

Given an m x n 2D binary grid representing '1' (land) and '0' (water), return the number of islands (groups of connected '1's, connected horizontally or vertically).

APPROACH & EXPLANATION

Scan every cell; whenever an unvisited '1' is found, it is the start of a new island, so increment the count and run a flood fill (DFS or BFS) from that cell to mark every connected '1' as visited (by mutating it to '0' or using a separate visited matrix), so it is not counted again. Each cell is visited a constant number of times, giving $O(m*n)$ time.

JAVA SOLUTION

```
class Solution {
    public int numIslands(char[][] grid) {
        if (grid == null || grid.length == 0) return 0;
        int count = 0;

        for (int r = 0; r < grid.length; r++) {
            for (int c = 0; c < grid[0].length; c++) {
                if (grid[r][c] == '1') {
                    count++;
                    dfs(grid, r, c);
                }
            }
        }
        return count;
    }

    private void dfs(char[][] grid, int r, int c) {
        if (r < 0 || r >= grid.length || c < 0 || c >= grid[0].length || grid[r][c] != '1') {
            return;
        }
        grid[r][c] = '0'; // mark visited
        dfs(grid, r + 1, c);
        dfs(grid, r - 1, c);
        dfs(grid, r, c + 1);
        dfs(grid, r, c - 1);
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def numIslands(self, grid: list[list[str]]) -> int:
        if not grid:
            return 0
        rows, cols = len(grid), len(grid[0])
        count = 0

        def dfs(r, c):
            if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] != '1':
                return
            grid[r][c] = '0'
            dfs(r + 1, c); dfs(r - 1, c); dfs(r, c + 1); dfs(r, c - 1)

        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == '1':
                    count += 1
                    dfs(r, c)

        return count
```

26. Group Anagrams

LeetCode #49

Hashing

PROBLEM STATEMENT

Given an array of strings `strs`, group the anagrams together. Return the groups in any order.

APPROACH & EXPLANATION

Two strings are anagrams if and only if their sorted character sequences are identical, so the sorted string makes a perfect hash-map key. Iterate through the input, sort each string's characters to form a canonical key, and append the original string to the list stored under that key in a hash map. Finally, return all the map's value lists. Sorting each string of length `k` costs $O(k \log k)$, so overall time is $O(n * k \log k)$.

JAVA SOLUTION

```
class Solution {
    public List<List<String>> groupAnagrams(String[] strs) {
        Map<String, List<String>> groups = new HashMap<>();

        for (String s : strs) {
            char[] chars = s.toCharArray();
            Arrays.sort(chars);
            String key = new String(chars);
            groups.computeIfAbsent(key, k -> new ArrayList<>()).add(s);
        }
        return new ArrayList<>(groups.values());
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def groupAnagrams(self, strs: list[str]) -> list[list[str]]:
        groups = defaultdict(list)
        for s in strs:
            key = ''.join(sorted(s))
            groups[key].append(s)
        return list(groups.values())
```

27. Best Time to Buy and Sell Stock

LeetCode #121

DP / Single Pass

PROBLEM STATEMENT

Given an array `prices` where `prices[i]` is the price of a stock on day `i`, find the maximum profit from choosing one day to buy and a later day to sell. Return 0 if no profit is possible.

APPROACH & EXPLANATION

Track the minimum price seen so far while scanning left to right. At each day, the best possible profit if selling today is `price[i] - minPriceSoFar`; update a running maximum profit with this value, then update `minPriceSoFar` for future days. This single pass runs in $O(n)$ time and $O(1)$ space.

JAVA SOLUTION

```
class Solution {
    public int maxProfit(int[] prices) {
        int minPrice = Integer.MAX_VALUE;
        int maxProfit = 0;

        for (int price : prices) {
            if (price < minPrice) {
                minPrice = price;
            } else if (price - minPrice > maxProfit) {
                maxProfit = price - minPrice;
            }
        }
        return maxProfit;
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def maxProfit(self, prices: list[int]) -> int:
        min_price = float('inf')
        max_profit = 0

        for price in prices:
            if price < min_price:
                min_price = price
            elif price - min_price > max_profit:
                max_profit = price - min_price
        return max_profit
```

28. Longest Substring Without Repeating Characters

LeetCode #3 [Sliding Window](#)

PROBLEM STATEMENT

Given a string *s*, find the length of the longest substring without repeating characters.

APPROACH & EXPLANATION

Use a sliding window with two pointers and a hash map storing the most recent index of each character. Expand the right pointer through the string; if the current character was seen before and its last index is within the current window, jump the left pointer to just after that last occurrence. Track the window length ($\text{right} - \text{left} + 1$) at each step and keep the maximum. This gives $O(n)$ time since each pointer moves forward monotonically.

JAVA SOLUTION

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        Map<Character, Integer> lastSeen = new HashMap<>();
        int left = 0, maxLen = 0;

        for (int right = 0; right < s.length(); right++) {
            char c = s.charAt(right);
            if (lastSeen.containsKey(c) && lastSeen.get(c) >= left) {
                left = lastSeen.get(c) + 1;
            }
            lastSeen.put(c, right);
            maxLen = Math.max(maxLen, right - left + 1);
        }
        return maxLen;
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        last_seen = {}
        left = 0
        max_len = 0

        for right, c in enumerate(s):
            if c in last_seen and last_seen[c] >= left:
                left = last_seen[c] + 1
            last_seen[c] = right
            max_len = max(max_len, right - left + 1)
        return max_len
```

29. Subarray Sum Equals K

LeetCode #560

Prefix Sum + HashMap

PROBLEM STATEMENT

Given an array of integers `nums` and an integer `k`, return the total number of continuous subarrays whose sum equals `k`.

APPROACH & EXPLANATION

Maintain a running prefix sum while scanning left to right, and a hash map counting how many times each prefix sum value has occurred so far. For the current prefix sum `S`, a subarray ending at the current index sums to `k` exactly when there was an earlier prefix sum equal to `S - k`; the count of such earlier prefixes (from the map) tells how many subarrays ending here sum to `k`. Add that count to the answer, then record the current prefix sum in the map. Initialize the map with `{0: 1}` to correctly count subarrays that start from index 0. $O(n)$ time.

JAVA SOLUTION

```
class Solution {
    public int subarraySum(int[] nums, int k) {
        Map<Integer, Integer> prefixCount = new HashMap<>();
        prefixCount.put(0, 1);

        int sum = 0, count = 0;
        for (int num : nums) {
            sum += num;
            count += prefixCount.getOrDefault(sum - k, 0);
            prefixCount.merge(sum, 1, Integer::sum);
        }
        return count;
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def subarraySum(self, nums: list[int], k: int) -> int:
        prefix_count = {0: 1}
        total = 0
        count = 0

        for num in nums:
            total += num
            count += prefix_count.get(total - k, 0)
            prefix_count[total] = prefix_count.get(total, 0) + 1
        return count
```

30. Sort Colors

LeetCode #75

Two Pointers (Dutch National Flag)

PROBLEM STATEMENT

Given an array `nums` with `n` objects colored red, white, or blue (represented as 0, 1, 2), sort them in place so that objects of the same color are adjacent, in the order red, white, blue, without using a library sort.

APPROACH & EXPLANATION

Use the Dutch National Flag algorithm with three pointers: `low` (boundary for 0s), `mid` (current element being examined), and `high` (boundary for 2s). Scan with `mid`: if `nums[mid]` is 0, swap it with `nums[low]` and advance both `low` and `mid`; if it's 1, just advance `mid`; if it's 2, swap it with `nums[high]` and decrement `high` (but don't advance `mid` yet, since the swapped-in value needs to be examined). This sorts the array in a single $O(n)$ pass with $O(1)$ extra space.

JAVA SOLUTION

```
class Solution {
    public void sortColors(int[] nums) {
        int low = 0, mid = 0, high = nums.length - 1;

        while (mid <= high) {
            if (nums[mid] == 0) {
                swap(nums, low++, mid++);
            } else if (nums[mid] == 1) {
                mid++;
            } else {
                swap(nums, mid, high--);
            }
        }
    }

    private void swap(int[] nums, int i, int j) {
        int temp = nums[i];
        nums[i] = nums[j];
        nums[j] = temp;
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def sortColors(self, nums: list[int]) -> None:
        low, mid, high = 0, 0, len(nums) - 1

        while mid <= high:
            if nums[mid] == 0:
                nums[low], nums[mid] = nums[mid], nums[low]
                low += 1
                mid += 1
            elif nums[mid] == 1:
                mid += 1
            else:
                nums[mid], nums[high] = nums[high], nums[mid]
                high -= 1
```

31. Binary Search

LeetCode #704

Binary Search

PROBLEM STATEMENT

Given a sorted array of integers `nums` and a target value, return the index of target if it exists in `nums`, or -1 if it does not. Must run in $O(\log n)$ time.

APPROACH & EXPLANATION

Maintain a search range `[lo, hi]`. Repeatedly check the middle element: if it equals the target, return its index; if it's smaller than the target, the target must be in the right half, so move `lo` past `mid`; if it's larger, move `hi` before `mid`. Continue until `lo` exceeds `hi`, meaning the target isn't present. Each iteration halves the search space, giving $O(\log n)$ time.

JAVA SOLUTION

```
class Solution {
    public int search(int[] nums, int target) {
        int lo = 0, hi = nums.length - 1;

        while (lo <= hi) {
            int mid = lo + (hi - lo) / 2;
            if (nums[mid] == target) {
                return mid;
            } else if (nums[mid] < target) {
                lo = mid + 1;
            } else {
                hi = mid - 1;
            }
        }
        return -1;
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def search(self, nums: list[int], target: int) -> int:
        lo, hi = 0, len(nums) - 1

        while lo <= hi:
            mid = lo + (hi - lo) // 2
            if nums[mid] == target:
                return mid
            elif nums[mid] < target:
                lo = mid + 1
            else:
                hi = mid - 1
        return -1
```

32. Invert Binary Tree

LeetCode #226

Tree

PROBLEM STATEMENT

Given the root of a binary tree, invert the tree (mirror it) and return its root.

APPROACH & EXPLANATION

Recursively swap the left and right children of every node. For the current node, first recurse into both children to invert their subtrees, then swap the (now-inverted) left and right references. The base case is a null node, which is trivially its own inverse. This is a simple postorder-style traversal, $O(n)$ time since every node is visited once.

JAVA SOLUTION

```
class Solution {
    public TreeNode invertTree(TreeNode root) {
        if (root == null) return null;

        TreeNode left = invertTree(root.left);
        TreeNode right = invertTree(root.right);

        root.left = right;
        root.right = left;

        return root;
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def invertTree(self, root: Optional[TreeNode]) -> Optional[TreeNode]:
        if not root:
            return None
        left = self.invertTree(root.left)
        right = self.invertTree(root.right)
        root.left, root.right = right, left
        return root
```

33. Same Tree

LeetCode #100

Tree

PROBLEM STATEMENT

Given the roots of two binary trees p and q, check whether they are structurally identical and the nodes have the same values.

APPROACH & EXPLANATION

Recursively compare both trees simultaneously. If both current nodes are null, they match. If only one is null, or their values differ, the trees are not the same. Otherwise, recursively check that the left subtrees match and the right subtrees match. This visits each node once, $O(n)$ time.

JAVA SOLUTION

```
class Solution {
    public boolean isSameTree(TreeNode p, TreeNode q) {
        if (p == null && q == null) return true;
        if (p == null || q == null) return false;
        if (p.val != q.val) return false;

        return isSameTree(p.left, q.left) && isSameTree(p.right, q.right);
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def isSameTree(self, p: Optional[TreeNode], q: Optional[TreeNode]) -> bool:
        if not p and not q:
            return True
        if not p or not q:
            return False
        if p.val != q.val:
            return False
        return self.isSameTree(p.left, q.left) and self.isSameTree(p.right, q.right)
```


D — Easy (Harder)

34. Maximum Depth of Binary Tree

LeetCode #104 Tree

PROBLEM STATEMENT

Given the root of a binary tree, return its maximum depth (the number of nodes along the longest path from the root down to the farthest leaf).

APPROACH & EXPLANATION

The depth of a tree rooted at a node is 1 (for the node itself) plus the larger of the depths of its left and right subtrees. This translates directly into a recursive function: the base case is an empty node with depth 0, and otherwise return $1 + \max(\text{depth}(\text{left}), \text{depth}(\text{right}))$. $O(n)$ time since every node is visited once.

JAVA SOLUTION

```
class Solution {
    public int maxDepth(TreeNode root) {
        if (root == null) return 0;
        return 1 + Math.max(maxDepth(root.left), maxDepth(root.right));
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def maxDepth(self, root: Optional[TreeNode]) -> int:
        if not root:
            return 0
        return 1 + max(self.maxDepth(root.left), self.maxDepth(root.right))
```

D — Easy (Harder)

35. Valid Parentheses

LeetCode #20 Stack

PROBLEM STATEMENT

Given a string `s` containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid: every opening bracket is closed by the same type of bracket, in the correct order.

APPROACH & EXPLANATION

Use a stack. Scan the string left to right: when an opening bracket is seen, push it. When a closing bracket is seen, check that the stack is non-empty and its top matches the corresponding opening bracket; if so, pop it, otherwise the string is invalid. At the end, the string is valid only if the stack is empty (every opener was matched). $O(n)$ time and space.

JAVA SOLUTION

```
class Solution {
    public boolean isValid(String s) {
        Deque<Character> stack = new ArrayDeque<>();
        Map<Character, Character> pairs = Map.of('(', ')', '[', ']', '{', '}');

        for (char c : s.toCharArray()) {
            if (pairs.containsKey(c)) {
                stack.push(c);
            } else if (pairs.containsValue(c)) {
                if (stack.isEmpty() || stack.pop() != pairs.get(c)) {
                    return false;
                }
            }
        }
        return stack.isEmpty();
    }
}
```

PYTHON SOLUTION

36. Two Sum

 LeetCode #1 [Hashing](#)

PROBLEM STATEMENT

Given an array of integers `nums` and an integer `target`, return the indices of the two numbers such that they add up to `target`. Assume exactly one solution exists, and the same element may not be used twice.

APPROACH & EXPLANATION

Use a hash map to store each number's index as you scan through the array. For each element, compute its complement (`target - current value`) and check whether the complement already exists in the map — if it does, those two indices are the answer. Otherwise, add the current number and its index to the map and continue. This single pass achieves $O(n)$ time compared to the $O(n^2)$ brute-force pair check.

JAVA SOLUTION

```
class Solution {
    public int[] twoSum(int[] nums, int target) {
        Map<Integer, Integer> seen = new HashMap<>();

        for (int i = 0; i < nums.length; i++) {
            int complement = target - nums[i];
            if (seen.containsKey(complement)) {
                return new int[] {seen.get(complement), i};
            }
            seen.put(nums[i], i);
        }
        throw new IllegalArgumentException("No two sum solution");
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def twoSum(self, nums: list[int], target: int) -> list[int]:
        seen = {}
        for i, num in enumerate(nums):
            complement = target - num
            if complement in seen:
                return [seen[complement], i]
            seen[num] = i
        raise ValueError("No two sum solution")
```

37. Palindrome Number

 LeetCode #9 [Math](#)

PROBLEM STATEMENT

Given an integer `x`, return `true` if `x` is a palindrome integer (reads the same forwards and backwards), without converting it to a string.

APPROACH & EXPLANATION

Negative numbers and numbers ending in 0 (except 0 itself) can never be palindromes, so handle those as immediate false. Otherwise, reverse the second half of the number by repeatedly extracting the last digit and building a reversed number, stopping once the reversed half becomes greater than or equal to the remaining first half. The number is a palindrome if the two halves match (accounting for an odd digit count by discarding the middle digit with `reversedHalf / 10`).

JAVA SOLUTION

```
class Solution {
    public boolean isPalindrome(int x) {
        if (x < 0 || (x % 10 == 0 && x != 0)) return false;

        int reversedHalf = 0;
        while (x > reversedHalf) {
            reversedHalf = reversedHalf * 10 + x % 10;
            x /= 10;
        }
        return x == reversedHalf || x == reversedHalf / 10;
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def isPalindrome(self, x: int) -> bool:
        if x < 0 or (x % 10 == 0 and x != 0):
            return False

        reversed_half = 0
        while x > reversed_half:
            reversed_half = reversed_half * 10 + x % 10
            x //= 10
        return x == reversed_half or x == reversed_half // 10
```

38. Reverse Linked List

LeetCode #206

Linked List

PROBLEM STATEMENT

Given the head of a singly linked list, reverse the list and return the new head.

APPROACH & EXPLANATION

Iterate through the list while keeping a "previous" pointer (initially null). For each node, save its next pointer before overwriting it to point backward to "previous", then advance both "previous" and the current pointer forward. When the current pointer becomes null, "previous" is the new head of the reversed list. $O(n)$ time, $O(1)$ space.

JAVA SOLUTION

```
class Solution {
    public ListNode reverseList(ListNode head) {
        ListNode prev = null;
        ListNode curr = head;

        while (curr != null) {
            ListNode next = curr.next;
            curr.next = prev;
            prev = curr;
            curr = next;
        }
        return prev;
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def reverseList(self, head: Optional[ListNode]) -> Optional[ListNode]:
        prev = None
        curr = head

        while curr:
            next_node = curr.next
            curr.next = prev
            prev = curr
            curr = next_node
        return prev
```

39. Contains Duplicate

LeetCode #217

Hashing

PROBLEM STATEMENT

Given an integer array nums, return true if any value appears at least twice in the array, and false if every element is distinct.

APPROACH & EXPLANATION

Scan through the array while adding each element to a hash set. Before adding, check if the element is already present in the set — if so, a duplicate has been found. If the scan completes without finding a repeat, all elements are distinct. $O(n)$ time, $O(n)$ space.

JAVA SOLUTION

```
class Solution {
    public boolean containsDuplicate(int[] nums) {
        Set<Integer> seen = new HashSet<>();

        for (int num : nums) {
            if (!seen.add(num)) {
                return true;
            }
        }
        return false;
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def containsDuplicate(self, nums: list[int]) -> bool:
        seen = set()
        for num in nums:
            if num in seen:
                return True
            seen.add(num)
        return False
```

40. Linked List Cycle

LeetCode #141

Two Pointers (Floyd's Algorithm)

PROBLEM STATEMENT

Given the head of a linked list, determine whether the linked list has a cycle in it.

APPROACH & EXPLANATION

Use Floyd's Tortoise and Hare algorithm: two pointers start at head, one (slow) moving one step at a time, the other (fast) moving two steps at a time. If there is no cycle, fast will reach the end (null) first. If there is a cycle, fast will eventually "lap" slow and they will meet at the same node, proving a cycle exists. This uses $O(1)$ extra space, unlike a hash-set-based visited check which needs $O(n)$ space.

JAVA SOLUTION

```
public class Solution {
    public boolean hasCycle(ListNode head) {
        ListNode slow = head, fast = head;

        while (fast != null && fast.next != null) {
            slow = slow.next;
            fast = fast.next.next;
            if (slow == fast) return true;
        }
        return false;
    }
}
```

PYTHON SOLUTION

```
class Solution:
    def hasCycle(self, head: Optional[ListNode]) -> bool:
        slow = fast = head

        while fast and fast.next:
            slow = slow.next
            fast = fast.next.next
            if slow == fast:
                return True
        return False
```